

01 Getting Started with Qt6

- 了解 Qt 的架构
- 了解 Qt 图形编程

- What is Qt ?

<https://www.qt.io/zh-cn/>

Qt Group

开发与设计 软件质量 解决方案 帮助与资源

🔍 ⓘ 🌐 学习中心

Price. Buy.

下载试用

📢 重磅消息: Qt Group 完成对 IAR 的战略收购

阅读更多 >

卓越软件，
由此启航

立即开始

登录 >

软件应具备易于开发、美观易用、设计可靠的特性。从日常设备到关键系统，我们的技术正重塑人类生活、出行和连接的方式。

驱动数十亿设备与应用

覆盖逾70个行业

服务数百万用户



What is Qt

Qt
开发框架
开发工具
AI助手
设计工具
支持的平台与语言
功能

设计、开发、部署 用户界面（UI）与应用程序

同一套代码库部署于嵌入式、桌面、移动端等目标平台。

免费试用

立即购买

开发框架

为UI开发、业务逻辑和机器间通信提供最完善的库集。

[了解更多 >](#)

开发工具

跨平台 IDE，拥有多种工具和扩展，提升 Qt 开发者的用户体验。

[了解更多 >](#)

设计工具

具备现成 UI 组件的可视化 2D/3D UI 编辑器。涵盖从原型设计到产品开发各阶段。

[了解更多 >](#)

<https://www.qt.io/qt-world>



Welcome to the Qt World

Here you can explore why Qt is the fastest and smartest way to produce industry-leading software that users love.

Explore now

开发框架(Framework)



The image shows the Qt Framework landing page. At the top, there is a navigation bar with links: Qt, 开发框架 (highlighted), 开发工具, AI助手, 设计工具, 支持的平台与语言, and 功能. The main heading is 'Qt开发框架' (Qt Framework) in large white characters, followed by '完整的软件开发框架' (Complete Software Development Framework) in green. Below this, a paragraph describes Qt as a complete, highly intuitive, modular C++ library that simplifies application development, generates high-quality, readable, maintainable, and reusable code, and has high runtime performance and low memory footprint. To the right, there is a collage of various Qt-based user interfaces on different devices. At the bottom, there is a search bar labeled '浏览' (Browse) and 'Your Qt Application/Device'.

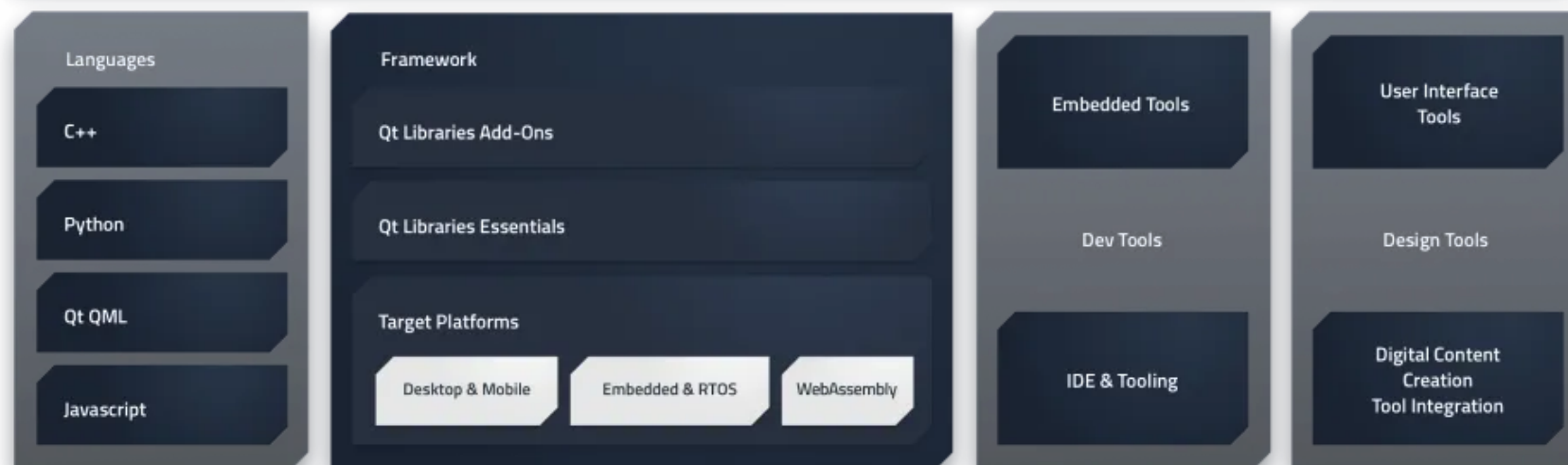
Qt 开发框架 开发工具 AI助手 设计工具 支持的平台与语言 功能

Qt开发框架

完整的软件开发框架

Qt开发框架包含一整套高度直观、模块化 的C++ 库类，并加载可简化应用程序开发的API。Qt 可生成高可读、易维护和可重用的代码，具有较高的运行时性能，且内存占用小——最重要的是，Qt是跨平台的。

浏览 Your Qt Application/Device



开发框架: Qt ESSENTIALS 和 Qt ADD-ONS

Qt ESSENTIALS

让真正的跨平台成为可能

Qt Essentials 定义了Qt在所有平台上的基本组件。在所有Qt支持的开发平台和经过测试的目标平台上都可用。

浏览Qt Essentials

Qt Core

其他模块使用的核心非图形类。

Qt GUI

图形用户界面组件的基类。包括 OpenGL。

Qt Quick

使用自定义用户界面构建高度动态的应用程序的声明性框架。

Qt Network

这些类使网络编程更容易和更可移植。

Qt Multimedia

音频、视频、广播和照相机功能的类。

Qt SQL

使用 SQL 进行数据库集成的类。

Qt Design Viewer : <https://designviewer.qt.io/>

QML Web : <https://qmlweb.github.io/>

Qt扩展模块

为合适的任务 选择正确的扩展模块

Qt Essentials 定义了Qt在所有平台上的基本组件，在所有Qt支持的开发平台和经过测试的目标平台上都可用。

浏览Qt 扩展模块

Qt OpenGL

OpenGL 支持类。不推荐使用 Qt GUI 模块中的 QOpenGL 类。

Qt Wayland Compositor

提供开发 Wayland 合成器的框架。

Qt WebView

通过使用平台固有的API在QML应用程序中显示Web内容，而无需包括完整的Web浏览器栈。

Qt Safe Renderer

提供UI渲染组件，可用于渲染功能安全系统中的安全关键项目，例如预警指标。

Qt Sensors

提供对传感器硬件和运动手势识别的接口。

Qt SCXML

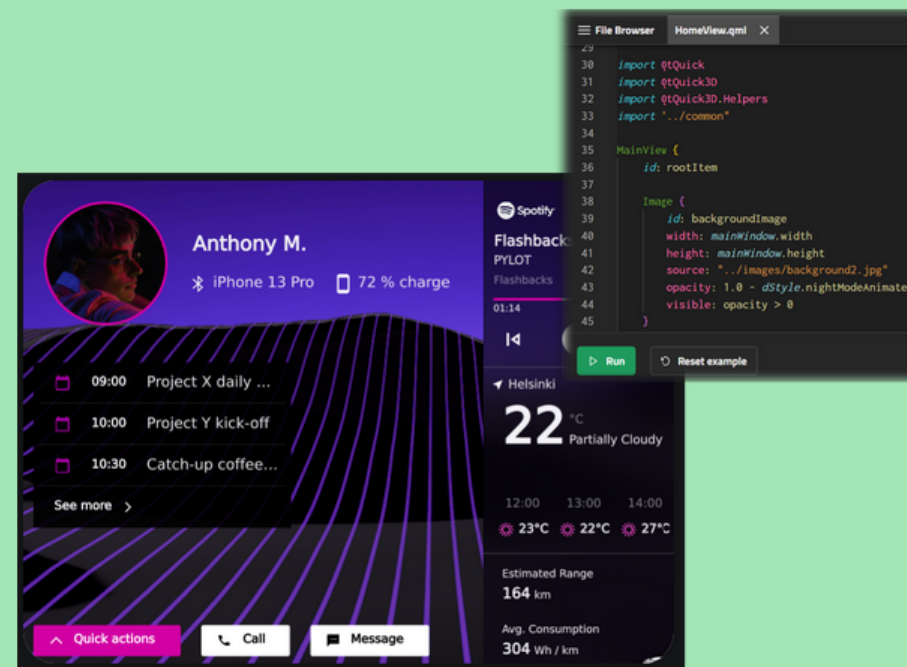
提供从 SCXML 文件创建状态机并将其嵌入应用程序的类和工具。

开发框架：Qt For WebAssembly

无需安装，直接在浏览器中体验Qt！

无需下载即可在浏览器中探索Qt的强大功能。测试、编辑演示示例，见证其潜力，并将您的想法变为现实！

在浏览器中试用



Qt Creator IDE

Qt

开发框架

开发工具

AI助手

设计工具

支持的平台与语言

功能



复杂的代码编辑器

您可以在Qt Creator高级代码编辑器上使用C ++、QML、JavaScript、Python和其他语言编写软件。它具备代码补全、语法突出显示、重构等功能，内置文档随手可得。



版本控制

Qt Creator集成了大多数流行的版本控制系统，包括Git、Subversion、Perforce 和 Mercurial。



项目和构建管理

无论导入现有项目，或是从头开始创建一个项目，Qt Creator都能生成所有必要文件。包括支持Cmake和用qmake构建工具进行交叉编译。



多种系统选择

在桌面、移动和嵌入式操作系统上构建并运行您的软件。构建设置使您可以轻松地在目标之间切换。



Qt设备模拟器

在与目标设备几乎相同的条件下测试、调试应用程序。



Qt Quick 编译器

将源代码编译为本机机器代码，加快启动时间和UI性能，并保护您的源代码和知识产权。

Qt Creator IDE的 GitHub Copilot 集成

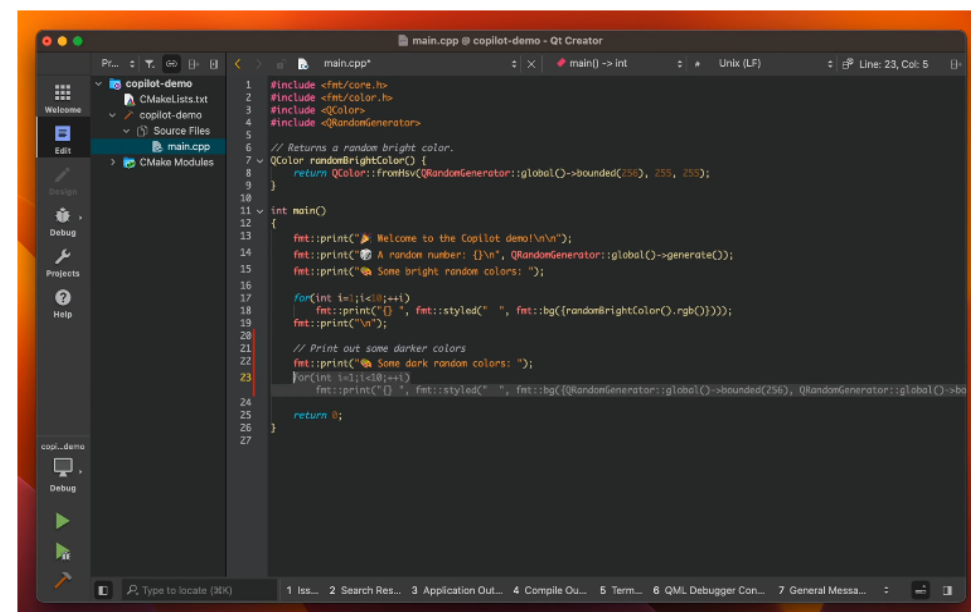
使用GitHub Copilot完成编码任务的速度最多可提供55%

借助GitHub Copilot集成^{*}，开发者可专注于复杂和创新的软件创建，而不是处理模板或重复代码。GitHub Copilot最多可将编码任务的完成速度提高55%。开发者可以从以下用例中的上下文感知建议中受益：

- 键入时自动完成代码
- 将注释转化为代码
- 基于现有代码创建测试用例
- 基于现有代码创建代码文档

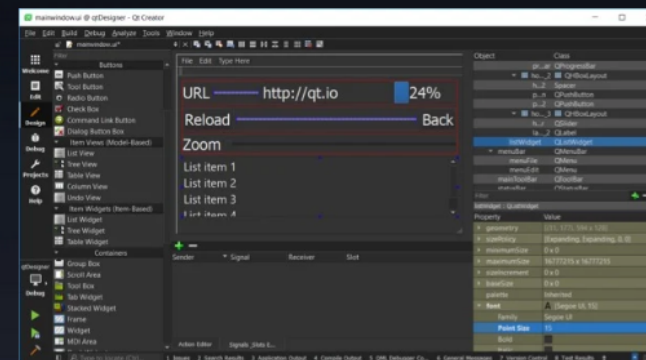
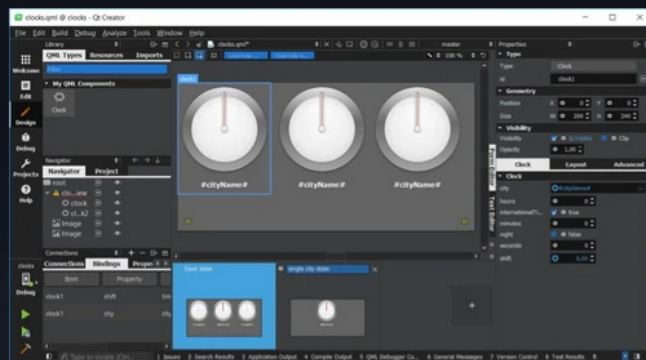
开发者可选择是否只查看来自有许可证代码建议。使用GitHub Copilot Business，可确保GitHub不保留提示和建议。有关GitHub Copilot的更多信息，请查看[GitHub页面](#)。有关GitHub Copilot的更多信息，请查看[GitHub页面](#)。

^{*}需要从GitHub单独订阅和许可



更多的开发工具

Qt Creator内部的UI构建工具



桌面性能分析器

性能分析器

通过时间线和火焰图直观显示CPU和RAM使用情况，快速识别性能瓶颈。

[分析 QML 应用程序 >](#)

测试和调试工具

自动检测单元测试，集成调试器支持代码优化器。

[Qt 测试 >](#)

[调试 >](#)

桌面性能分析器

构建和编译工具

借助 Qt 对编译器和构建系统的广泛支持，在桌面、移动和嵌入式目标上的不同平台部署应用程序。

[添加编译器 >](#)

[指定构建设置 >](#)

最终用户文档

使用内置的搜索功能生成嵌入应用程序中的交互式最终用户文档。

[Qt Help框架 >](#)

国际化和本地化

设计适用于各种语言和地区的适应性软件应用程序，无需工程更改。添加特定区域的组件，例如日期、时间和数字格式。

[技术文档 >](#)

向导和示例

在 Qt 项目向导的引导下，立即设置您的项目。借助大量示例、代码示例和分步教程，可以更快入门。

[创建项目 >](#)

[Qt 示例和教程 >](#)

Qt

开发框架

开发工具

AI助手

设计工具

支持的平台与语言

功能

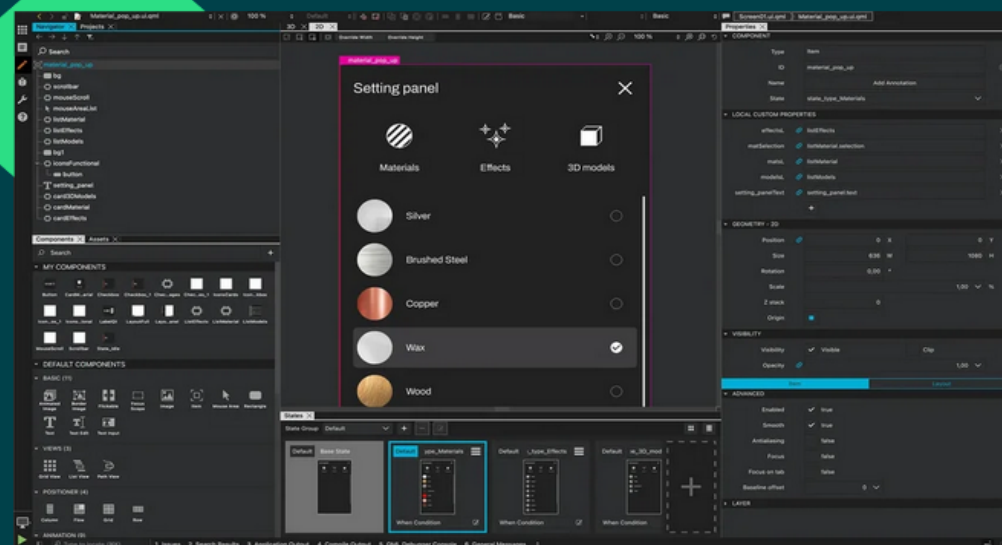
Qt Design Studio

让您的设计栩栩如生

通过弥合设计师与开发者之间的鸿沟，将您的设计愿景转化为可运行的 UI，从而变革您的开发流程。

免费试用

立即购买



超越 基础原型开发

当谈论打造UI时，我们不仅指探讨用现有素材创建模型。您可以利用Qt Design Studio让素材转变为可运行的 UI 构成。

自动 生成代码

使用 Figma、Adobe XD 或 Adobe Photoshop 等设计工具创建UI设计。将它们导入 Qt Design Studio，代码就会自动生成供开发者使用。

随时随地 测试与运行

模拟并验证交互和动态行为。Qt Design Studio构建的所有代码均可跨平台，编译到任何硬件或操作系统环境中。

https://www.bilibili.com/video/BV1uz4y1h7S1/?spm_id_from=888.80997.embed_other.whitelist

支持的平台和语言

支持多种平台

在 Linux、Windows 或 Mac 上开发，并在各种操作系统、浏览器或裸机上部署。

[查看是否支持您的平台 >](#)



支持多种参考目标设备

易于开发针对多种不同操作系统和硬件平台的应用。Qt可用于各种嵌入式目标设备。

[了解哪些是现成的参考镜像，以及如何制作自己的镜像 >](#)

[需要用微控制器? 快来试用Qt for MCUs >](#)

[了解用Qt为嵌入式设备设计和开发软件 >](#)

[查看Qt for Device Creation支持的目标设备 >](#)



支持多种语言

Qt的C++ API提供了易于跨平台开发的功能。使用Qt QML设计用户界面，这是一种声明式语言，能用JavaScript描述逻辑业务。如果您更喜欢Python或其他语言，我们拥有多种语言绑定。

[C++文档 >](#)

[Python文档 >](#)

[Qt QML文档 >](#)

[看看Qt for Python能做什么 >](#)

Languages supported by the Qt Company



A couple of the languages backed by the community



Qt Features



请选择软件包、许可证模式、平台或编程语言，以查看哪些项目可用于该选项。

请注意，不支持多选。

请注意，开源许可证并不限制工具的运行。例如，GPLv3确认运行未经修改的软件的无限权限。许可证设置了创建和分发衍生作品的条件。使用GPL v3许可的Qt Creator并不意味着您的软件获得了GPL v3许可，除非您的软件修改了您将要分发的现有软件，并且这些软件是根据GPL v3许可的。

Search

Search



SDLC Core Activity

Design

Develop

Deploy

Qt Editions

Qt Community Edition



License Models

LGPL v3



Development Platforms

Linux



Target Platforms

Select a target platform



Coding Language Support

Select a programming language



Clear filters ✕

开发工具

Qt 拥有自己的跨平台集成开发环境（IDE），并且内置了大量工具，专为开发应用程序和用户界面（UI）而设计，能够一次开发，部署到多个操作系统上。

Qt Creator

A cross-platform, complete integrated development environment (IDE) for application developers to create applications for multiple desktop, embedded, and mobile device platforms.

Qt Linguist

A set of tools for translating Qt C++ and Qt Quick applications into local languages.

qmake

Qt's build automation tool that simplifies the build process across different platforms. Although qmake is still supported, the official build tool for Qt is CMake and we recommend developers to use CMake in new projects.

CMake

CMake is a tool to simplify the build process for development projects across different platforms. CMake automatically generates build systems, such as Makefiles and Visual Studio project files.

makeqpf

A tool to generate pre-rendered fonts for embedded Linux.

Meta-Object Compiler (moc)

Generates meta-object information for QObject subclasses.

User Interface Compiler (uic)

Generates C++ code from user interface files.

Resource Compiler (rcc)

Embeds resources into Qt applications during the build process.

Qt D-Bus XML compiler (qdbusxml2cpp)

Converts D-Bus interface descriptions to C++ source code.

D-Bus Viewer

Introspection tool for D-Bus objects and messages.

Qt Visual Studio Tools

Integrates the Qt development tools into Microsoft Visual Studio.

Qt Distance Field Generator

Generates distance fields to improve load times of text-heavy Qt Quick user interfaces.

框架基础

这些是为Qt提供核心支撑的API和库。Qt包含丰富的基础组件，用于支持更高级的用户界面和应用程序开发。

Search



SDLC Core Activity

[Design](#)
[Develop](#)
[Deploy](#)

Qt Editions

[Qt Community Edition](#)


License Models

[LGPL v3](#)


Development Platforms

[Linux](#)


Target Platforms

[Select a target platform](#)


Coding Language Support

[Select a programming language](#)


Qt Core



Core non-graphical classes used by other modules.

Qt GUI



Base classes for graphical user interface components. Includes OpenGL.

Qt Network



Classes to make network programming easier and more portable.

Qt QML



Classes for QML and JavaScript languages.,All,All,C++

Qt Quick Dialogs



Types for creating and interacting with system dialogs from a Qt Quick application.

Qt Quick Layouts



Layouts are items that are used to arrange Qt Quick 2 based items in the user interface.

Qt D-Bus



Classes for inter-process communication over the D-Bus protocol.

Qt Quick



A declarative framework for building highly dynamic applications with custom user interfaces.

Qt Quick Controls



Provides lightweight QML types for creating performant user interfaces for desktop, embedded, and mobile devices. These types employ a simple styling architecture and are very efficient.

Qt Quick Test



A unit test framework for QML applications, where the test cases are written as JavaScript functions.

Qt Test



Classes for unit testing Qt applications and libraries.

Qt Widgets



Classes to extend Qt GUI with C++ widgets.

Search

Search



SDLC Core Activity

Design

Develop

Deploy

Qt Editions

Qt Community Edition



License Models

LGPL v3



Development Platforms

Linux



Target Platforms

Select a target platform



Coding Language Support

Select a programming language



Clear filters ✕

框架附加组件

Qt是一个功能极其丰富的框架，除了基础功能外，还提供了众多附加功能，旨在为您提供真正专业的开发体验。

Qt Multimedia



Classes for audio, video, and camera functionality.

Qt Multimedia Widgets



Qt Multimedia sub-module, containing widget-based classes for implementing multimedia functionality.

Active Qt



Classes for applications which use ActiveX and COM.

Qt 3D



Functionality for near-realtime simulation systems with support for 2D and 3D rendering.

Qt Bluetooth



Provides access to Bluetooth hardware.

Qt Concurrent



Classes for writing multi-threaded programs without using low-level threading primitives.

Qt Help



Classes for integrating documentation into applications, similar to Qt Assistant.

Qt Image Formats



Plugins for additional image formats: TIFF, MNG, TGA, WBMP.

Qt NFC



Provides access to Near-Field communication (NFC) hardware.

Qt OpenGL



OpenGL support classes. Deprecated in favor of the QOpenGL* classes in the Qt GUI module.

Qt Positioning



Provides access to position, satellite and area monitoring classes.

Qt Print Support



Classes to make printing easier and more portable.

Qt Quick Widgets



Provides a C++ widget class for displaying a Qt Quick user interface.

Qt Remote Objects



Provides an easy to use mechanism for sharing a QObject's API (Properties/Signals/Slots) between processes or

Qt SCXML



Provides classes and tools for creating state machines from SCXML files and embedding them in applications.

01 Getting Started with Qt6

- What is Qt ?
- Qt6 概览

欢迎使用 Qt 6

面向未来的生产力平台



下一代用户体验

使用设计师和开发者友好型工具、更精简但功能更强大的QML语言以及灵活的图形一贯体验。

Latest Release – Qt 6.10

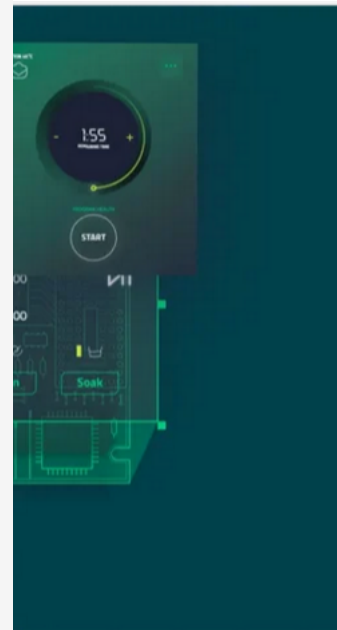
This release highlights:

无限的可扩展性

从微控制器 (MCU) 到超级计算机, QML 设计、开发和部署软件所需的一切, 创建前端美观、后端强大的应用

- Out-of-the-box accessibility capabilities,
- Easy imports for vector animations,
- Building UIs easily with the new Flexbox layout, search field and QML types,
- Much more to keep your product modern.

发现无限的可扩展性资源



[Release notes](#) > [Release blog post](#) > [Release video](#) >

Qt6 亮点

Qt 6亮点

Qt渲染硬件接口

Direct 3D、Metal、Vulkan和OpenGL。只需编写一次渲染代码，即可部署在任何硬件上。

Qt Quick 3D

整合2D和3D内容到一个技术栈上。

Qt Quick Controls 2桌面样式

像素级完美、原生外观的控件无缝集成入操作系统。

HiDPI支持

独立缩放的支持，针对不同的显示器配置自动缩放UI。

QProperty系统

通过C++中的绑定支持提高代码速度，将QML最好用部分带入Qt，并与QObject无缝集成。

并发API的改进

多核CPU、并行计算、保持用户界面流畅的同时在后台执行后端逻辑。自动根据硬件进行线程数量管理。

网络功能的改进

创建您自己的通信后端，并将其集成到默认的Qt工作流中，自动添加与安全性相关的功能。

3D微粒系统

在 Qt Quick 3D 或 Qt Design Studio 中自定义您的3D微粒。

Qt Creator中的Qt覆盖测试

分析C++和QML代码的测试和代码覆盖率。

顶点动画

使用**网格变形**模拟软材料的变形或使用**骨骼动画**抽象表示模型的运动

CMake支持

凭借行业标准构建系统、丰富的功能集以及庞大的生态体系构建Qt应用程序。

Qt for Microcontrollers (MCUs)

轻量级渲染引擎可在具有2D硬件加速的低成本硬件上部署基于QML的UI，从而以最小的占用空间（> 80KB内存）实现最佳图形性能。

更新到C++17

更新到最新标准，提高代码的可读取性，更好性能和更容易维护。

Qt6 亮点

IDE与工具

Qt Creator IDE

Qt 6准备就绪

CMake支持得到了彻底改进和完善。

插件使用的改进

安装插件向导可简化扩展插件的构建和使用。

为C++20做好准备

更新代码模型以支持C++20功能。

开发者体验

多个快捷键可执行相同操作，以交互方式启动Qt for Python，改善了项目加载的响应速度。

其他改进

支持Catch2测试框架。

Qt Design Studio 2.0

Qt 6下一代2D和3D UI开发工具

UI设计和开发工具，使设计师和开发者能快速实现原型设计、开发复杂的用户界面。

Figma Bridge

除了从Sketch和Photoshop进行2D导入外，您现在还可以从Figma导入资产。

缩略图生成

自定义组件会自动生成带有工具提示和完整预览的图标，从而更容易获得整体效果。

Item旋转

使用新的旋转工具改善了直观的项目旋转。

导航器改进

锁定完整的项目，以免意外编辑。为匹配常用图形设计工具，增加Item的逆序排列。隐藏了未激活的图标，减少了视觉混乱，并更容易发现组件关系。

Qt for MCUs

改进支持。

开发主机操作系统

Qt 6.8支持的操作系统

Windows

Windows 10 (1809 或更高版本) (x64)

Windows 11 (x64 和 Arm)

Apple

macOS 10.5或更高版本 (64位Intel、64位ARM; XCode 12 或更高版本)

Linux

Ubuntu 22.04 (x86_64, gcc 11)

Ubuntu 24.04 (x86_64、arm64、gcc 13)

OpenSuse 15.5 (x86_64, gcc 9 至 12)

SLES 15 SP5 (Linux Enterprise Server 15, x86_64, gcc10)

Red Hat 8.6和8.8 (x86_64, gcc 10)

Red Hat 9.2 (x86_64、gcc 11)

Debian 11.6 (x86_64、arm64、gcc 10)

支持的目标操作系统

Windows

适用于ARM的Windows 10和11

Apple

macOS 12至15

Linux

带有Yocto 4.0和Yocto 5.0的嵌入式Linux(Scarthgap)

Red Hat 8.6、8.8和9.2

openSUSE 15.5

SUSE Linux Enterprise Server 15 SP5

基于ARM的Ubuntu 22.04和Debian 11.6（技术预览版）

Debian 12（企业软件包）

实时操作系统（RTOS）

完整性 19.0.13

VxWorks 7 (24.03)

FreeRTOS（Qt for MCUs）

QNX 7.1和8.0（技术预览版）

嵌入式Android和Android Automotive OS

Android Automotive OS 10 至 13

嵌入式 Android：从Android 11(SDK 30)到Android 13 (SDK 33)

LG webOS 2.26

移动端

Android 9 (API 28) 至 14(API 34)。

iOS 16至18

Web浏览器

使用Qt for WebAssembly将Qt应用程序部署到Web浏览器上

云端

Amazon Web Services（Yocto 5.0）

AUTOSAR Classic支持

适用于Renesas RH850/D1M1A汽车平台的Qt for MCUs的AUTOSAR平台移植

Qt 6在线社区

[Forum](#)

[Qt Project LinkedIn group](#)

[Qt mailing lists](#)

[Qt Discord Server](#)

[Qt Framework Subreddit](#)

[C++/Qt Telegram group](#)

[Qt Embedded Telegram group](#)

[YouTube Channel](#)

[Qt LinkedIn](#)

[Qt Project on Twitter](#)

[Qt Facebook](#)

[Qt Project Facebook](#)

[Stackoverflow tag](#)

[Wikipage for online communities](#)



现已推出Qt 6 QML电子书

在线书籍

这本在线电子书展示了使用全新Qt 6开发应用程序的方方面面，重点介绍了Qt Quick 技术，并包括关于为 Qt Quick 编写C++后端和扩展的必要信息。

阅读愉快！

Qt.io

Qt Development

Offering ▾ Solutions ▾ Resources ▾ Support ▾



Developers

Price. Buy.

Download. Try.

Qt Demos

From immersive 3D experiences to concrete industrial applications, our demos showcase the art of possible with Qt Group's technologies.

Access some of our demos online to experience first-hand Qt's capabilities and industry-grade use cases.

[Book a Live Demo](#)

Day Mode



Lights



Color



Environment

0.16

Swing

0.40

Boom

0.65

Arm

0.42

Bucket



<https://www.qt.io/development/qt-demos>

For Developers

Explore our thoughtfully curated collection of developer resources that are conveniently located, all in one place.



Try Qt Without Installation—Right in Your Browser!

Explore Qt's power in your browser with no downloads. Test and edit demos, see its potential, and bring your ideas to life!

[Try in Browser](#) >



Qt Academy

Start your learning journey into unlocking your full Qt potential.

[Learn More](#) >

<https://www.qt.io/development/developers>

The Future of Digital Experiences

Welcome to the documentation pages for **Qt**, the cross-platform software development framework.

Get Qt

[Get and install Qt](#) >[Build Qt from source](#) >[Qt Debian packages \(Technology Preview\)](#) >

Framework

[Qt modules](#) >[Additional libraries](#) >[Long-term support releases](#) >

Get started

[New to Qt](#) >[Port from 5 to 6](#) >[API reference](#)[C++](#) > [QML](#) > [Python](#) >

Tools

[Qt Creator](#) >[Design Studio](#) >[Quality Assurance](#)[Squish](#) > [Coco](#) > [Test Center](#) >[All tools](#) >

Embedded

[Qt for MCUs](#) >[Boot to Qt](#) >[Qt for Android Automotive](#) >[All embedded topics](#) >[All Qt Documentation](#) >

External Resources

Qt Wiki

Community-maintained Qt articles

[Qt Wiki](#) >

Qt Forum

Discussion board for Qt-related topics

[Qt Forum](#) >

Bug Reports

Bug tracker for Qt and related projects

[Bug Reports](#) >

Code Review

Review tool for contributing to Qt

[Code Review](#) >

ACADEMY

Learn Qt development with Qt Academy

Interactive courses, tutorials, and learning paths for Qt developers

RESOURCES

Learn what you can achieve with Qt

Webinars, tutorials, videos, and more



Qt6模块

- **Qt Essentials** : 定义了 Qt 在所有平台的基础, 在所有支持的 **开发平台** 和已测试的 **目标平台** 上都可使用。除了正在测试的那些模块 (为了保持源码兼容), 这些模块在整个 Qt6 中都保持源码和二进制兼容。
 - × **基本模块**对大多数 Qt 应用是通用的和有用的。
 - + 如果某个模块用于**某个特定目的**, 即使在所有平台都可用, 但仍被视为**扩展模块**。
- **Qt Add-Ons** : 为特定目的带来额外的价值, 通常仅仅在某个开发平台上有效。
 - × 许多扩展模块或者出于功能完整和向后兼容而存在, 或者仅仅用于某些平台。
 - × 每个扩展模块独立指定自己的兼容性承诺。

Qt 基本模块

Qt Essentials 定义了 Qt 在所有平台上的基础。它们在所有支持的开发平台和测试的目标平台上都可用。除了测试模块将保持源代码兼容外，这些模块将在整个 Qt 6 中保持源代码和二进制兼容。

基本模块是通用模块，对大多数 Qt 应用程序都有用。用于特殊目的模块被视为附加模块，即使它在所有支持的平台上都可用。

Qt Core

其他模块使用的核心非图形类。

Qt D-Bus

通过 D-Bus 协议进行进程间通信的类。

Qt GUI

图形用户界面（GUI）组件的基础类。

Qt Network

使网络编程更简单、更可移植的类。

Qt Qml

QML 和 JavaScript 语言类

Qt Quick

声明式框架，用于构建具有自定义用户界面的高动态应用程序。

Qt Quick Controls

轻量级 QML 类型，用于为桌面、嵌入式和移动设备创建性能良好的用户界面。

Qt Quick Dialogs

用于从 Qt Quick 应用程序创建系统对话框并与之交互的类型。

Qt Quick Layouts

布局是用于在用户界面中安排基于 Qt Quick 2 的项目的项目。

Qt Quick Test

QML 应用程序的单元测试框架，测试用例以 JavaScript 函数的形式编写。

Qt Test

用于 Qt 应用程序和库单元测试的类。

Qt Widgets

用 C++ widget 扩展 Qt GUI 的类。

Qt Essentials define the foundation of Qt on all platforms. They are available on all supported development platforms and on the tested target platforms. Except the testing modules, which will remain source compatible, these modules will remain source and binary compatible throughout Qt 6.

Qt 附加模块

Addon modules bring additional value for specific purposes. These modules may only be available on some of the development platforms.

Qt 附加模块可为特定用途带来额外价值。这些模块可能只适用于某些开发平台。许多附加模块要么功能完备，存在向后兼容性，要么只适用于某些平台。每个附加模块都会单独说明其兼容性承诺。

Qt 安装程序包含下载附加组件的选项。更多信息，请访问[Qt 入门](#)页面。

Active Qt

使用 ActiveX 和 COM 的应用程序类（仅限 Windows）。

Qt Bluetooth

提供对蓝牙硬件的访问。

Qt CoAP

实现 RFC 7252 定义的 CoAP 客户端。

Qt Concurrent

用于编写多线程程序的类，无需使用底层线程原语。

Qt Graphs

提供以条形图、散点图和曲面图等三维形式以及以面积图、条形图、甜甜圈图、线图、饼图、散点图和样条图等二维形式显示数据的功能。

Qt GRPC

提供从用于与 gRPC® 服务通信的 Qt Protobuf 规范生成基于 Qt 的类的功能。

Qt Help

将文档集成到应用程序中的类

Qt HTTP Server

用于将 HTTP 服务器嵌入 Qt 应用程序的框架。

Qt Image Formats

用于其他图像格式的插件：tiff、mng、tga、wbmp。

Qt Lottie Animation

用于渲染 JSON 格式图形和动画的 QML API，由 Adobe® After Effects 的 Bodymovin 插件导出。

Qt MQTT

提供 MQTT 协议规范的实现。

Qt Multimedia

丰富的 QML 类型和 C++ 类用于处理多媒体内容。还包括处理摄像头访问的 API。

Qt Network 授权

为基于 OAuth 的在线服务授权提供支持。

Qt NFC

提供对近场通信（NFC）硬件的访问。在桌面平台上，NDEF 访问仅支持类型 4 标签。

Qt OPC UA

工业应用中的数据建模和数据交换协议。

Qt OpenGL

使 Qt 应用程序轻松使用 OpenGL 的 C++ 类。一个单独的库（Qt OpenGL Widgets）提供了一个用于渲染 OpenGL 图形的部件。

Qt Positioning

提供对位置、卫星信息和区域监控类的访问。

Qt Print Support

使打印更方便、更便携的类别。

Qt Protobuf

提供从 Protobuf 规范生成基于 Qt 的类的功能。

Qt Quick 3D

为创建基于 Qt Quick.NET 的 3D 内容或用户界面提供高级 API。

Qt Quick 3D 物理学

Qt Quick 3D Physics 提供了一个高级 QML 模块，为 Qt Quick 3D 添加了物理模拟功能。

Qt Quick 小工具

提供用于显示 Qt Quick 用户界面的 C++ widget 类。

Qt Quick Effects

提供 QML 类型，用于向 Qt Quick 项目应用一种或多种简单的 Graphical Effects。

Qt Quick Particles

为粒子效果提供 QML 类型。

Qt Quick Timeline

支持基于关键帧的动画和参数化。

Qt Remote Objects

为在进程或设备之间共享 QObject 的 API（属性/信号/插槽）提供了一种易于使用的机制。

Qt SCXML

提供从 SCXML 文件创建状态机并将其嵌入应用程序的类和工具。

Qt Sensors

提供对 Android、iOS 和 Windows 平台上传感器硬件的访问。

Qt Serial Bus

提供对串行工业总线接口的访问。目前，该模块支持 CAN 总线和 Modbus 协议。

Qt Serial Port

提供与硬件和虚拟串行端口交互的类。

QtShader Tools

为跨平台 Qt Shader 管道提供工具。这些工具可处理图形和计算着色器，使其可用于 Qt Quick 和 Qt 生态系统中的其他组件。

Qt Spatial Audio

提供空间音频支持。在三维空间中创建包含不同声源和房间相关属性（如混响）的声音场景。

Qt SQL

使用 SQL 进行数据库集成的类

技术预览版中的 Qt 附加组件

以下附加组件仍在开发中，但可作为技术预览版使用：

Qt Location



提供 QML 和 C++ 接口，用于创建位置感知应用程序。

Qt 扩展模块

Qt 扩展模块为 Qt 带来了额外的功能。它们可能是某个 Qt 版本的一部分，但在发布时间、平台支持、Qt 提供的二进制文件、商业支持或兼容性承诺等方面可能与其他 Qt 模块有所不同。您可以在每个扩展模块的模块文档中找到具体细节。

Qt WebEngine



用于在使用 Chromium 浏览器项目的应用程序中嵌入网页内容的类和函数。

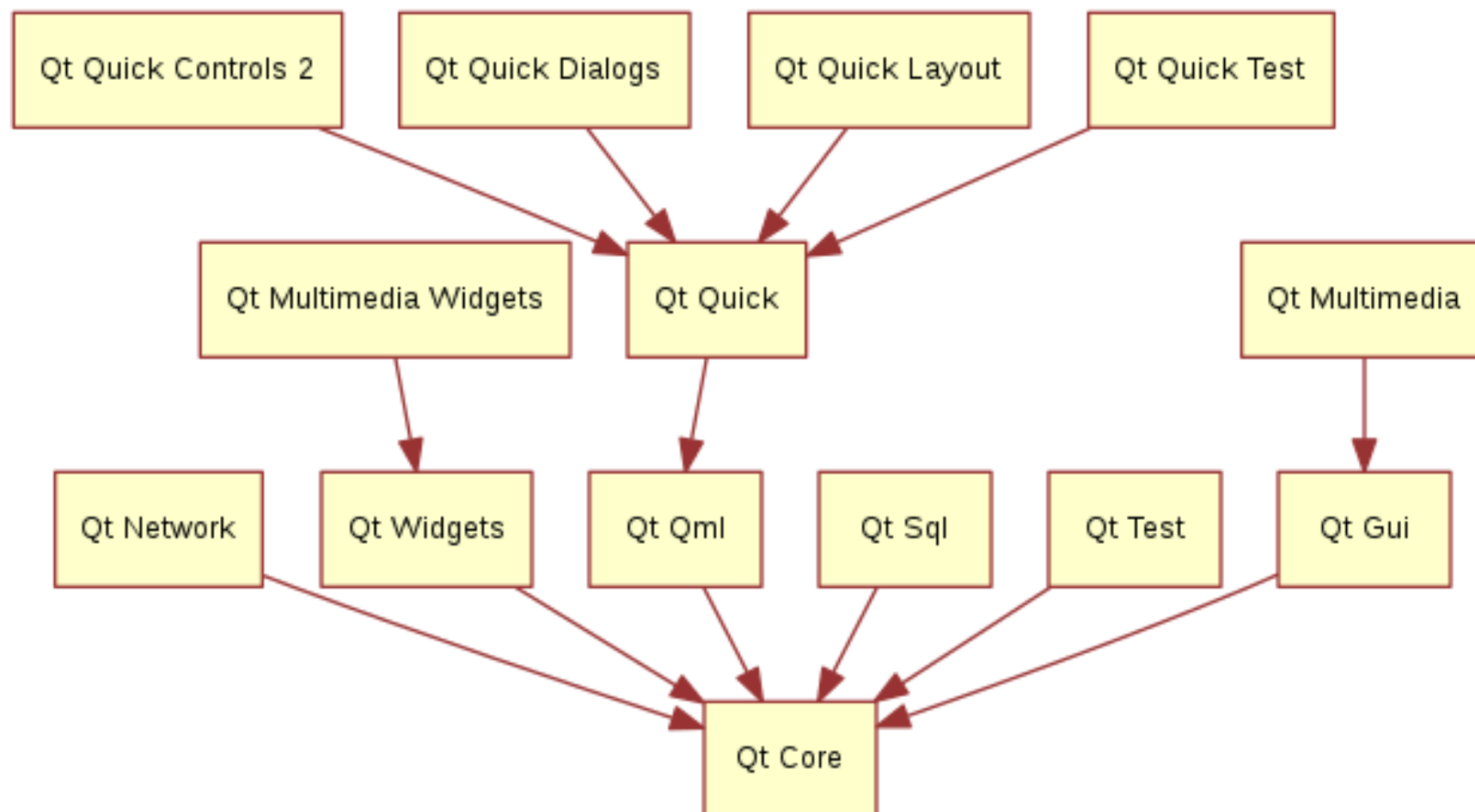
Qt PDF



用于在桌面平台上渲染 PDF 文档的类和函数。

Qt6 Core 模块

Qt Core 模块是所有其他 Qt 模块的基础。



Qt Core 对 C++ 的扩展

The Qt Core module adds these features to C++:

- › a very powerful mechanism for seamless object communication called signals and slots
- › queryable and designable object properties
- › hierarchical and queryable object trees that organize object ownership in a natural way with guarded pointers (**QPointer**)
- › a dynamic cast that works across library boundaries

The following pages provide more information about Qt's core features:

- › **The Meta-Object System**
- › **The Property System**
- › **Object Model**
- › **Object Trees & Ownership**
- › **Signals & Slots**

<https://doc.qt.io/qt-6/qtcore-index.html>

Qt Core 模块

- Qt Core module 加入了以下功能特性到 C++:
 - x 信号与槽：一种非常强大的对象间无缝通信机制
 - x 对象 properties：可查询和可识别的 object properties
 - x 层次化和可查询的对象树：该对象树通过守护指针 (QPointer) 以一种自然的方式组织了对象的所有权。
 - x 动态转换：一种可以跨越库的边界工作的动态转换机制
- 关于这些核心特性的更详细信息参见下面的参考页：
 - x The Meta-Object System(最基础)
 - x The Property System
 - x Object Model(利用 C++ 继承 QObject 类和使用 Meta-Object System 机制)
 - x Object Trees & Ownership
 - x Signals & Slots

Qt 模块的使用

- 使用某个 Qt 模块需要链接到对应的模块库文件，可以直接链接或者通过其他依赖链接。
- 构建工具为模块的使用和链接提供了支持，包括 CMake and qmake，此时需要在项目文件中说明需要使用和链接的 Qt 模块：
 - × qmake 构建 Qt 项目会默认链接 Qt core 模块，但链接其他模块需要在 .pro 项目文件中给 QT 变量添加模块名称，例如：QT+=widgets
 - × 使用 CMake 构建时，需要使用 find_package() 命令来定位在 Qt6 包中模块的组件：

```
find_package(Qt6 REQUIRED COMPONENTS Core)
target_link_libraries(mytarget PRIVATE Qt6::Core)
```

Threading and Concurrent Programming

- Qt Core 以平台独立的线程类的形式提供了线程支持、提供了一种事件传递的线程安全方式，以及跨线程的若干信号与槽连接。
 - × 多线程编程典型的应用是用于执行耗时操作，使得应用用户界面不会失去响应。
 - × 注意，Qt 的线程类是使用原生线程 APIs 实现的，例如 Win32 和 pthreads 。因此，它们可以与具有相同原生 API 的线程一起使用。
- Qt 中提供了 4 种不同的方法来实现多线程应用：
 - × QThread: Low-Level API with Optional Event Loops
 - × QThreadPool and QRunnable: Reusing Threads
 - × Qt Concurrent: Using a High-level API
 - × WorkerScript: Threading in QML

Input/Output, Resources, and Containers

Qt provides a resource system for organizing application files and assets, a set of containers, and classes for receiving input and printing output.

- › Container Classes
- › Serializing Qt Data Types
- › Implicit Sharing

In addition, Qt Core provides a platform-independent mechanism for storing binary files in the application's executable.

- › The Qt Resource System

Additional Frameworks

Qt Core also provides some of Qt's key frameworks.

- › The Animation Framework
- › JSON Support in Qt
- › CBOR Support in Qt
- › Inter-Process Communication
- › How to Create Qt Plugins
- › The Event System
- › Application Permissions

元对象系统

- 元对象系统是一个 C++ 扩展，使得 C++ 更适合真正的组件 GUI 编程
- 元对象系统提供了用于对象间通信的信号与槽机制、运行时类型信息和动态 property 系统。
- 元对象系统基于三个事物：
 - × **QObject 类**，即为使用元对象系统的对象所提供的基类；
 - × **Q_OBJECT 宏**，位于类声明的私有区域，用于启用元对象特性，如动态 property，信号和槽；
 - × **元对象编译器 (moc)**，为每个 QObject 的子类提供了实现元对象特性所需的代码。

对象模型、对象树

- Qt 对象模型提供了 OO 编程的可能性，它基于两种技术实现：
 - × 对象模型的许多功能特性，都是基于对 QObject 的继承，通过标准的 C++ 技术，来实现的。部分相关的功能特性如下：
 - + 强大的事件和事件过滤器
 - + 用于国际化的上下文字符串翻译
 - + 复杂的内部驱动定时器，使得在事件驱动的 GUI 中可以优雅地集成许多任务
 - × 其他的功能特性（如信号与槽、动态 property 系统、对象树），则通过 moc 所支持的元对象系统来实现。
- 所有的 QObject 对象以对象树来组织自身。当您使用另一个对象作为父对象创建一个 QObject 对象时，该对象将自动地将自己添加到父对象的 children() 列表中。父对象获得该对象的所有权；也就是说，它将自动地在析构函数中删除它的所有孩子对象。

QObject 提供的功能

- QObject 是所有 Qt 类的基类，由元对象模型提供，是对象模型的核心
 - x 提供了信号与槽机制的静态函数： `connect()/disconnect()`
 - x 该类的所有函数都是可重入的
 - x 提供了支持对象树机制的成员函数： `findChildren()` 和 `parent()`
 - x 每个对象都有一个 `objectName()`，它的类名可以通过相应的 `metaObject()` 找到 (参见 `QMetaObject::className()`)
 - x 当一个对象被删除时，它会发出一个 `destroyed()` 信号。您可以捕捉这个信号以避免对 QObjects 的悬空引用。
 - x 提供了事件处理函数的接收事件的函数接口 `event()`
 - x 提供了 property 机制的函数 `property()` <https://doc.qt.io/qt-6/qobject.html>
 - x 提供了基本的计时器支持函数 `startTimer()`

01 Getting Started with Qt6

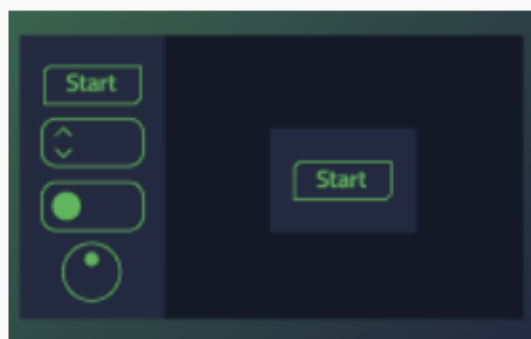
- What is Qt ?
- Qt6 概览
- Qt6 文档
- Qt6 图形编程概述

Qt 入门

你好，欢迎使用 Qt!

<https://doc.qt.io/qt-6/gettingstarted.html>

要开始使用 Qt，我们需要知道您想做什么。



设计用户界面

- › 用于Qt Quick UI 的工具
- › 安装Qt Design Studio
- › 使用教育许可证安装Qt Design Studio
- › 获取并安装 Qt
- › 支持的平台
- › 创建您的第一个应用程序
- › 探索 Qt

```
1 // main.cpp
2 #include <QApplication>
3 #include <QTableView>
4 #include "myModel.h"
5
6 int main(int argc, char *arg[])
7 {
8     QApplication a(argc, argv);
9     QTableView table/new;
10    MyModel myModel;
11 }
12
```

开发应用程序

- › 应用程序开发解决方案
- › 获取并安装 Qt
- › 使用教育许可证安装 Qt
- › 支持的平台
- › 创建您的第一个应用程序
- › 探索 Qt

Solutions for UI Design

- Qt Quick(Qt User Interface Creation Kit) 界面是动态的，带有流畅的动画，专门为触摸界面进行了优化。
- Qt Widgets 则与桌面的观感紧密集成。

您可以使用目标平台的原生观感 (look and feel) 创建 Qt Quick 和 Qt Widgets 界面。

UI 方案比较: <https://doc.qt.io/qt-6/topics-ui.html>

作为需要为应用程序设计用户界面的设计师、技术艺术家或开发人员，您需要做出一些决定。请看下面的列表，了解它们在多大程度上符合您应用程序的要求，以及我们推荐的用户界面技术。

如果您的应用程序	如果您的应用程序
› 使用短周期进行原型开发。 › 在移动或嵌入式设备或 MCU 上运行。 › 在触摸屏上运行。 › 包含动画和图形效果。	› 大型标准行业应用。 › 仅在桌面上运行。 › 看起来像本地 Linux、macOS 和 Windows 应用程序。 › 与底层平台集成良好。
我们建议使用 Qt Quick.	我们建议使用 Qt Widgets.

Qt Widgets Application

- Qt Widgets 是桌面环境中典型的用户界面元素。利用这些 widgets 可以编写与底层平台集成良好的 **Qt Widgets Application**，并在 Linux、Windows 和 macOS 上提供了原生观感。
 - × 这些 widgets 非常成熟、功能丰富，主要适用于大多数传统的 user interfaces。相对于 Qt Quick，这些 widgets 适合创建大型的桌面应用，不太适合创建以触摸为中心的，具有流畅界面的应用。
 - × Qt Widgets 模块的使用：<https://doc.qt.io/qt-6/qtwidgets-index.html>
 - + qmake 项目中在 pro 文件中添加：QT += **w**idgets
 - + cmake 项目，在 cmake 文件中添加：
 - ✓ find_package(**Qt6** REQUIRED COMPONENTS **W**idgets)
 - target_link_libraries(mytarget PRIVATE **Qt6::W**idgets)

widgets 及其基类 QWidget

- QWidget 类是所有 widgets 的基类，它提供了渲染屏幕以及处理用户输入事件的基本能力。
 - × 通过继承 QWidget 或适当的 QWidget 子类并重新实现 virtual event handlers，可以创建自定义 widgets。

<https://doc.qt.io/qt-6/qtwidgets-index.html>

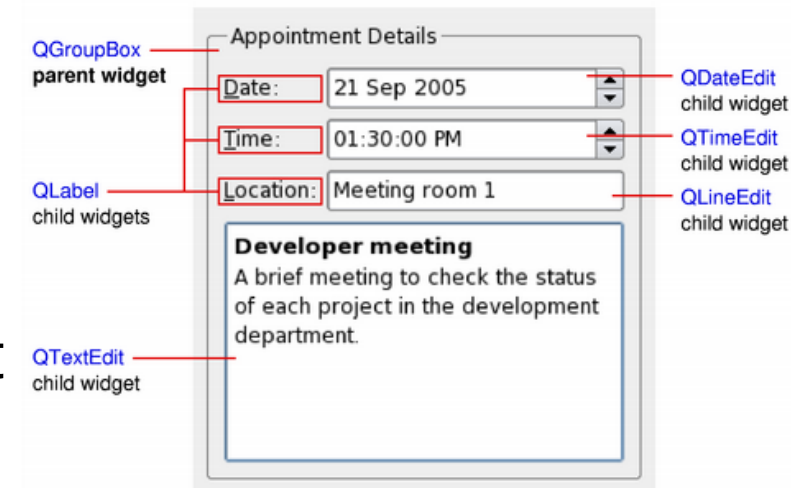
- widgets 可以：
 - <https://doc.qt.io/qt-6/qwidget.html>
 - <https://doc.qt.io/qt-6/widget-classes.html#the-widget-classes>

- × 显示数据和状态信息，接收用户输入
- × 为，应该组合在一起的，其他 widgets 提供一个容器，形成对象树。

每个 widget 都可指定一个唯一的父 widget

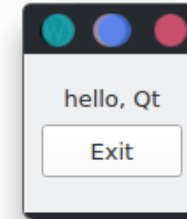
- + window：没有指定父 widget 的顶层

widget 对象。参见第 1 章中的所有例程



Qt Widgets 程序与 object tree

```
1  #include <QLabel>
2  #include <QPushButton>
3  #include <QApplication>
4
5  int main(int argc, char *argv[])
6  {
7      QApplication a(argc, argv);
8
9      QWidget window; // 顶层widget对象, 其父指针实参为默认值nullptr
10     window.setGeometry(400, 300, 100, 90);
11
12     auto label = new QLabel("hello, Qt", &window); // 创建window对象的一个孩子label
13     label->setAlignment(Qt::AlignCenter);
14     label->setGeometry(10, 10, 80, 30);
15
16     auto button = new QPushButton("Exit", &window); // 创建window对象的一个孩子button
17     button->setGeometry(10, 40, 80, 30);
18
19     QObject::connect(button, &QPushButton::clicked, &a, &QApplication::quit);
20
21     window.show();
22
23     return a.exec();
24 }
```



Qt Widgets 编程：基于 Class 的 C++ OOP

- 静态语言 (如 C++、Java、C# 等) 通常采用基于 Class 的 OOP :
 - × 先定义某个 **Class** 来描述同一类对象的共性；再将其做为对象的模板 (即 **Class type**) 在源程序中创建类的变量来请求创建对象。
 - + 源码中**对象**的创建语句被编译为二进制版本，然后直接在硬件机器上执行。创建语句每执行一次，就按**模板及其初始式**创建一个对象，确定其初始状态；但创建之后不允许在运行时修改对象的结构：只能修改对象的**数据值**，不能动态增加 / 删除**数据或方法成员**
 - Qt 元对象系统扩展了 C++，因此 Qt 程序允许在运行时修改**对象**的结构
 - × 静态语言会提供许多已经描述好的类，放在一起形成**类库**。
- 基于类的 **OOP** 通常用于表示业务领域中的实体对象及其关联，通常需要**信息隐藏**；而 Qt 提供的，是不需信息隐藏的基于类的 **OOP**，通常用于表示 UI 层 / 数据管理层的系统边界对象。

Qt Quick Application

- Qt Quick 技术提供了 **QML 及其库**，用于创建动态流畅的 UI。
- 这些 UI 既可以作为 **UI Prototype(application)**；也可以与完成计算的 **JavaScript/C++ 代码后端**相结合，形成一个 **Qt Quick Application**。

The following list contains related overviews:

- › **Qt Quick Controls**
- › **QML Applications** - introduction to QML
- › **Visual types in QML**
- › **Responding to User Input in QML**
- › **Animations in QML**
- › **Displaying Text in QML**
- › **Layouts in QML**
- › **Style and Theme Support**
- › **Integrating JavaScript in QML**
- › **Qt Quick 3D** - add-on for 3D graphics
- › **Qt Shader Tools** - add-on shader effects

QML、QML 基础库、QML 标准库

- **QML(Qt Modeling Language)**：一种用户界面描述和编程语言，使用它可以创建高性能、流畅动画的、有视觉吸引力的 Qt Quick Applications。
 - × QtQML 模块提供了 QML 语言和引擎基础设施 (C++ API)，还提供了一些在应用中使用的**基础 QML Types**(QML API)，组成 **QML 基础库**。
 - + 可通过导入 **QtQml** QML 模块来使用 QML 基础库，即 **import QtQml;**
- **QML 标准库**：提供大量可视化的，带有动画的**标准 QML 类型**。
 - × Qt Quick 模块是编写 QML 程序的**标准库**。它提供了一个可视化画布，包括用于创建和动画化可视化组件、接收用户输入、创建数据模型和视图以及延迟对象实例化的类型。Qt Quick 模块提供的
 - + C++ API 可用来通过 C++ 扩展 QML 应用；
 - + QML API 组成了 QML 标准库，使用时需要导入 QtQuick **QML 模块**，即：**import QtQuick;**

Qt Quick 扩展库

<https://doc.qt.io/qt-6/zh/qtquick-qmlmodule.html>

- **QML 扩展库**：一组 controls、对话框和布局类型等，用于构造完整 UI
 - × 这些扩展类型位于 Qt Quick 若干子模块 (Controls/Dialog/Layout 等) 的 QML API 中，它们组成了 **QML 扩展库**。
 - + 其中的 controls 有几种内建的**风格**，也支持自定义风格。
 - × 使用**扩展库**需要导入相应的子模块：**import QtQuick.Controls**、**import QtQuick.Dialog**、**import QtQuick.Layout**

Qt Quick includes several submodules that contain additional types:

- > **Local Storage** - a submodule containing a JavaScript interface for an SQLite database
- > **Particles** - provides a particle system for QML applications
- > **Controls** - provides a set of reusable UI components
- > **Layouts** - contains types that are used to arrange items in the user interface
- > **Tests** - types for testing QML applications.
- > **Effects** - provides types for applying one or more simple graphical effects to Qt Quick items.
- > **Shapes** - provides types for rendering vector shapes in a Qt Quick scene.
- > **Vector Image** - provides types for displaying vector image files.

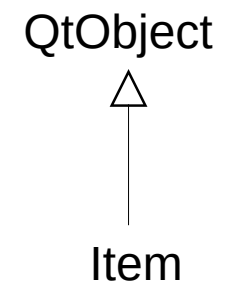
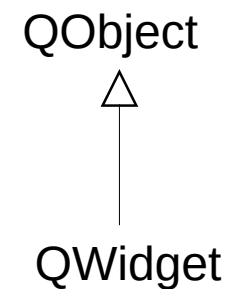
QML 对象类型及 QObject 基类型

- QML 对象类型是创建 QML UIs 的基本元素。其中，QObject 是所有 QML 对象类型的基 (对象) 类型。
 - × QObject 是一个不可见的元素，实际上它就是一个 QObject 对象，只包含 objectName property。
 - × QObject 可用来构造新的轻量级对象，例如创建一个 QObject 对象并添加一组 custom properties，可构造出一个轻量级的新对象。

```
import QtQuick

Item {
    QObject {
        id: attributes
        property string name
        property int size
        property variant attributes
    }

    Text { text: attributes.name }
}
```



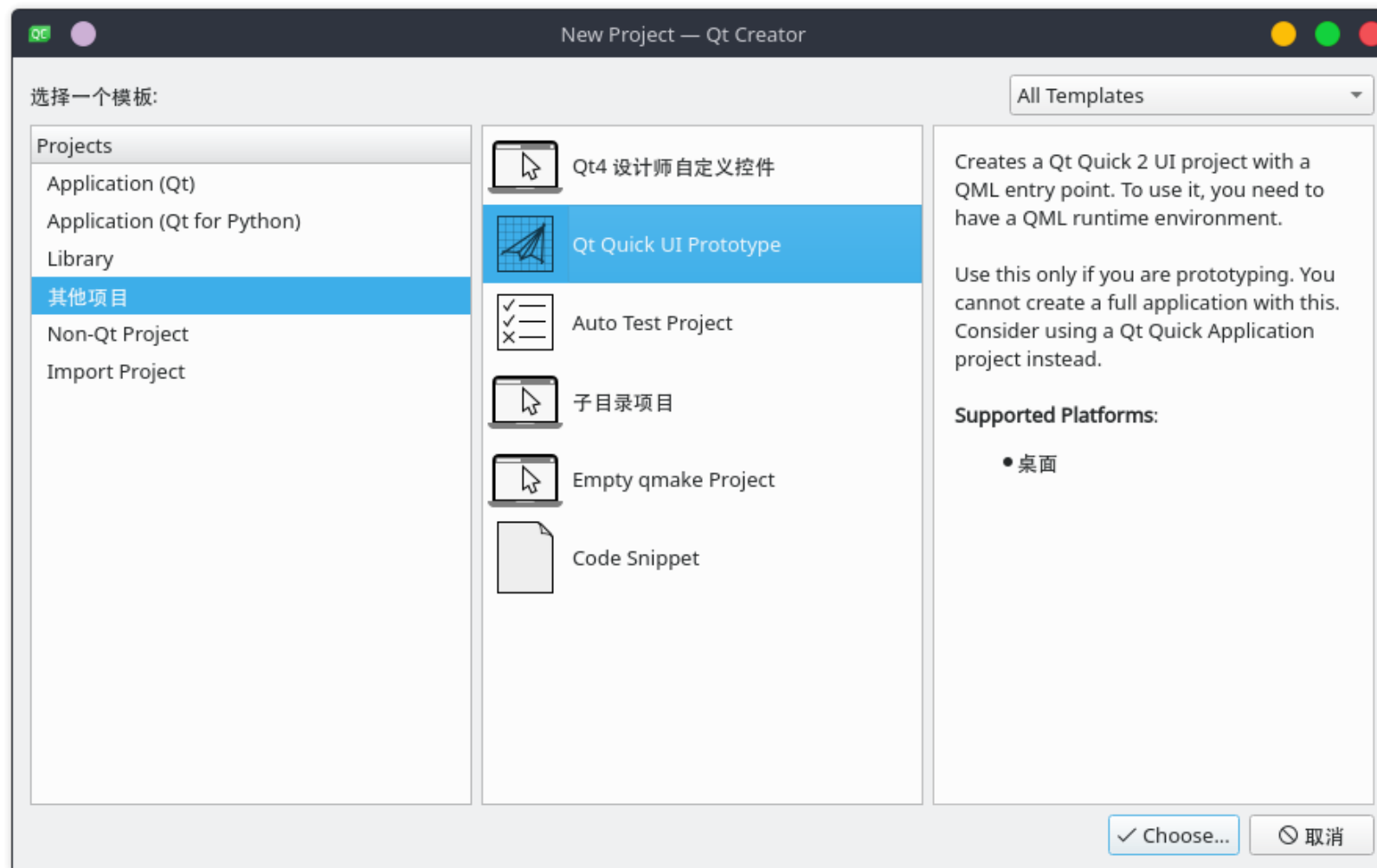
Item 及其子对象类型

- Item type 是 QML 标准库中所有可视项 (visual items) 的基类型。
 - × Item 类型继承自 QObject，本身也是不可见的，但是它定义了它的孩子可视项的通用 properties，如 x, y 坐标，width 和 height 等；同时，还可以作为可视项的容器使用。
 - × Item 类型实际上是一个 QQuickItem C++ 对象，QQuickItem 类继承自 QObject 类，用于构造 QML 的 object types 层次结构。
 - × 下面是继承自 Item 的子 object types：

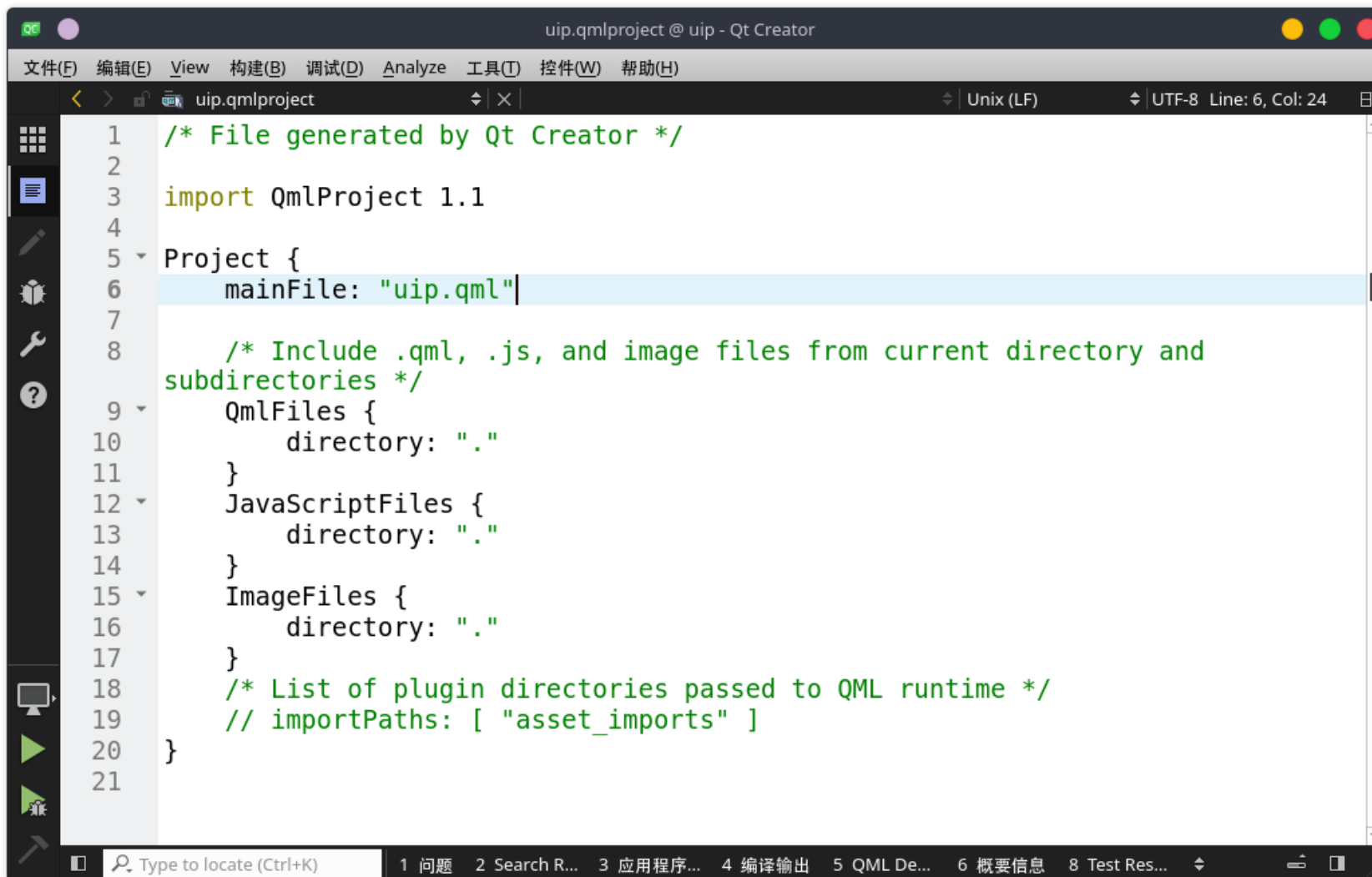
AnimatedSprite, BorderImage, Canvas, Column, ColumnLayout, DropArea, Flickable, Flipable, Flow, FocusScope, Grid, GridLayout, Image, LayoutItemProxy, Loader, MouseArea, MultiEffect, MultiPointTouchArea, ParticlePainter, PathView, PinchArea, Rectangle, Repeater, Row, RowLayout, ShaderEffect, ShaderEffectSource, Shape, SpriteSequence, StackLayout, Text, TextEdit, TextInput, VectorImage, and WindowContainer

Qt Quick UI Prototype

QML UI 可以通过只包含 QML 代码的 UI Prototype 项目来体验；该类项目不需要编译，只需使用工具 QML Runtime(即 QML 解释器) 来加载运行。



Qt Quick UI Prototype 的项目配置文件



- UI Prototype 的项目配置文件是后缀为 `.qmlproject` 的文件，这里是 `uip.qmlproject` 文件：文件里面项目被当作一个对象进行配置

Qt Quick UI Prototype 的对象树

- Qt Quick UI Prototype 项目是纯粹的 QML 程序，需要使用 QML 运行时程序来解释执行。其 UI 界面的根对象可以是任意的 QML item 对象
 - × QML Runtime 在解释执行 UI Prototype 时，提供了默认的顶层窗口对象，源码中的根对象（如基于 Rectangle 的对象）被寄宿在该窗口对象中。

```
import QtQuick
import QtQuick.Controls

Rectangle {
    width: 400
    height: 400
    visible: true

    Button {
        id: button
        anchors.centerIn: parent
        text: "OK"
    }
}
```

QML at a Glance

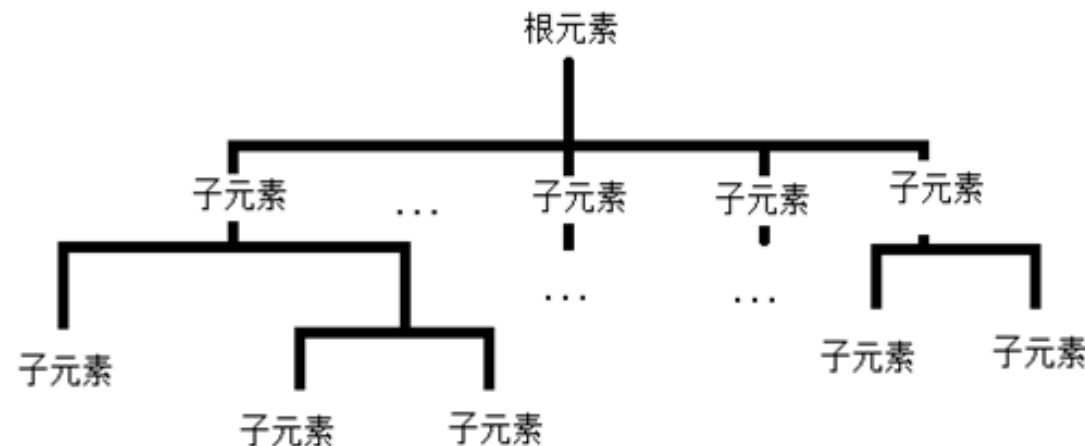
```
> meetqml.qml Item
1 import QtQuick
2
3 Item {
4     id: window
5     width: 200; height: 100
6     Text {
7         id: helloText
8         anchors.horizontalCenter: parent.horizontalCenter
9         color: "red"
10        text: "MEET QML!"
11        function resetTextPosition() {
12            anchors.verticalCenter = parent.verticalCenter
13        }
14    }
15    Image {
16        id: helloImage
17        anchors.horizontalCenter: parent.horizontalCenter
18        anchors.top: helloText.bottom
19        source: "qt_logo.png"
20    }
21    TapHandler{
22        onPressed: {
23            helloImage.visible = false
24            helloText.text = "Bye-bye picture!"
25            helloText.color = "green"
26            helloText.resetTextPosition()
27        }
28    }
29 }
30
```



该例程基于 Item、Text、Image 和 TapHandler 基对象 (类型) 分别创建了 4 个对象 (1 个新的 Item 对象、1 个新的文本对象、1 个图片对象和 1 个触摸处理器对象)。

QML GUI 的对象布局和坐标系

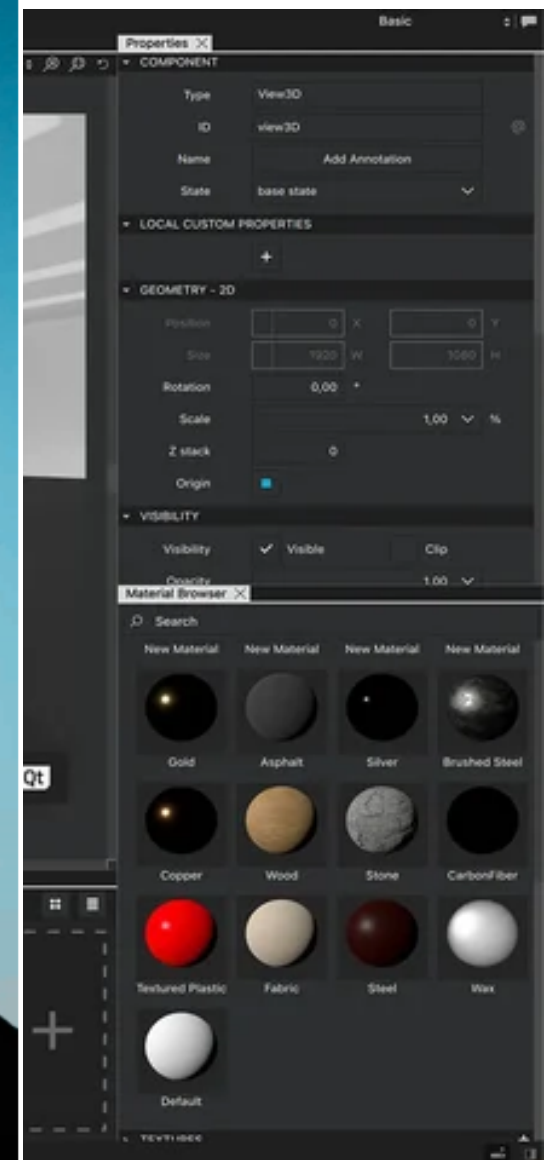
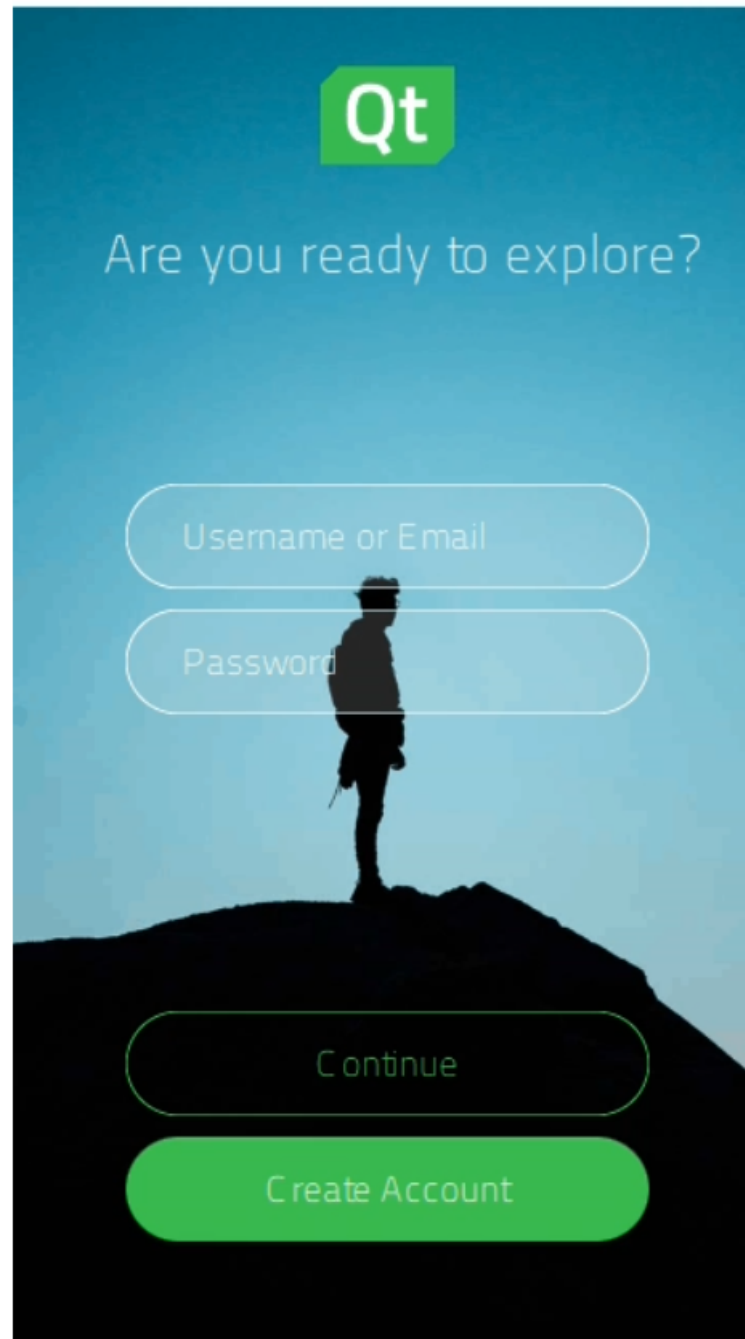
- QML GUI 中的所有图形对象也组织为一个对象树：
 - × 最顶层是一个根对象 (元素)，里面可以嵌套一个或多个子对象，子对象里面还可以包含子对象。
 - × QML 对象树，类似于 Widgets 对象树，是 QML GUI 的对象机制
- QML 根对象的屏幕坐标系，原点在屏幕左上角，x 轴从左向右增大，y 轴从上到下增大，z 轴从屏幕向外增大。
 - × 子对象从父对象上继承了坐标系统，它的 x，y 总是相对于它的父对象坐标系。



Qt Quick 编程：基于 Object 的 QML OOP

- 动态语言 (如 Javascript、QML 等) 通常采用基于 Object 的 OOP :
 - ✗ 先直接描述一些有名的基础对象，将这些对象作为 base Objects (不描述对象的共性，只需描述各类对象的基本样品，也称为 Object Type)，来描述新对象即可。这里没有类变量的概念，因为你直接描述的就是一个新对象。
 - + 源程序描述的对象通过解释器来解释执行，不必编译：解释器根据对象的描述去直接创建对象，也允许在运行时改变对象的结构。
 - ✗ 语言中许多典型的 object 描述，放在一起形成对象模块，称为对象库
 - + 动态语言创建对象时，会直接基于某个有名 base object 的描述来描述新对象；新对象在解释执行时，才在系统中创建。
 - ✓ 新对象的描述，不仅可以修改 base object 的数据值，还允许动态地增加或删除数据或方法成员，在解释执行时就会创建出一个基于 base object，但又非常容易扩展的新对象，具有很大的灵活性。
- 基于 Object 的 OOP 通常用于表示 UI 层边界对象，不需信息隐藏

QML/C++(JavaScript) 混合编程的 Qt Quick Application



meetqml Qt Quick Application 的 main.cpp 模板

```

1  #include <QGuiApplication>
2  #include <QQmlApplicationEngine>
3
4  int main(int argc, char *argv[])
5  {
6      QGuiApplication app(argc, argv);
7
8      QQmlApplicationEngine engine;
9      QObject::connect(
10         &engine,
11         &QQmlApplicationEngine::objectCreationFailed,
12         &app,
13         []() { QCoreApplication::exit(-1); },
14         Qt::QueuedConnection);
15     engine.loadFromModule("meetqml", "Main");
16
17     return app.exec();
18 }
19

```

```

1  cmake_minimum_required(VERSION 4.2.3)
2
3  #https://github.com/Kitware/CMake/blob/v4.2.3/Help/dev/experimental.rst
4
5  set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD "d0edc3af-4c50-42ea-a356-e2862fe7a444")
6  set(CMAKE_CXX_MODULE_STD ON)
7
8  project(meetqml VERSION 0.1 LANGUAGES CXX)
9
10 set(CMAKE_AUTOMOC ON)
11 set(CMAKE_CXX_STANDARD_REQUIRED ON)
12
13 find_package(Qt6 REQUIRED COMPONENTS Quick)
14 qt_add_executable(appmeetqml)
15 qt_add_qml_module(appmeetqml
16     URI meetqml
17     VERSION 1.0
18     SOURCES    main.cpp
19     QML_FILES  Main.qml
20 )
21
22 target_sources(appmeetqml
23     PRIVATE
24     PRIVATE FILE_SET cxx_modules TYPE CXX_MODULES FILES
25         #math.cppm # 接口单元
26 )
27
28 target_compile_features(appmeetqml PRIVATE cxx_std_23)
29 target_link_libraries(appmeetqml
30     PRIVATE
31     Qt6::Quick
32 )
33
34 set_target_properties(appmeetqml PROPERTIES
35     AUTORCC TRUE # set(CMAKE_AUTORCC ON)
36     # MACOSX_BUNDLE_GUI_IDENTIFIER com.example.appmeetqml
37     MACOSX_BUNDLE_BUNDLE_VERSION ${PROJECT_VERSION}
38     MACOSX_BUNDLE_SHORT_VERSION_STRING ${PROJECT_VERSION_MAJOR}.${PROJECT_VERSION_MINOR}
39     MACOSX_BUNDLE TRUE
40     WIN32_EXECUTABLE TRUE
41 )

```

- main.cpp 中创建了一个 QGuiApplication 对象，用于管理 Qt Quick Application 的所有对象和资源（类似于 Widget Application 中的 QApplication 对象）。
- Main.qml 中必须定义一个 Window 对象，用于控制应用窗口的大小、标题等。

Qt Quick Application 的对象树

- Qt Quick Application 要求 QML 界面的根对象必须是一个基于 **Window** 或其子类型 (如 **ApplicationWindow**) 的对象。

```
import QtQuick

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")
}
```

```
1 import QtQuick
2 import QtQuick.Controls
3
4 ApplicationWindow {
5     id: window
6     visible: true
7     width: 200
8     height: 100
9     title: qsTr("Hello QML")
10
11     Text {
12         id: helloText
13         anchors.horizontalCenter: parent.horizontalCenter
14         text: "MEET QML!"
15         function resetTextPosition() {
16             anchors.verticalCenter = parent.verticalCenter
17         }
18     }
19     Image {
20         id: helloImage
21         anchors.horizontalCenter: parent.horizontalCenter
22         anchors.top: helloText.bottom
23         source: "icons/qt_logo.png"
24     }
25     TapHandler{
26         onPressed: {
27             helloImage.visible = false
28             helloText.text = "Bye-bye picture!"
29             helloText.resetTextPosition()
30         }
31     }
32 }
```

Window 类型概述

- Window 对象会为某个 Qt Quick scene(场景) 创建一个新的顶层窗口。它会自动地为使用 Qt Quick 图形类型自动地配置窗口。
- 一个 Window 可以声明在一个 Item 或另一个 Window 的内部；此时，内部的 Window 将自动地寄宿于外部窗口：即，大多数平台默认会在外部窗口的中央显示内部窗口。
 - × 如果内嵌窗口想成为应用的对话框，可以设置标志为 Qt.Dialog 。
因为有些窗口管理器没有这个标志的话，不会提供居中的行为。
- 在一个顶层的 QObject 中也可以声明多个窗口，此时，这些窗口没有寄宿管理。
- 你可以设置或者绑定 x 或 y 属性，以便显式地在屏幕上定位 Window

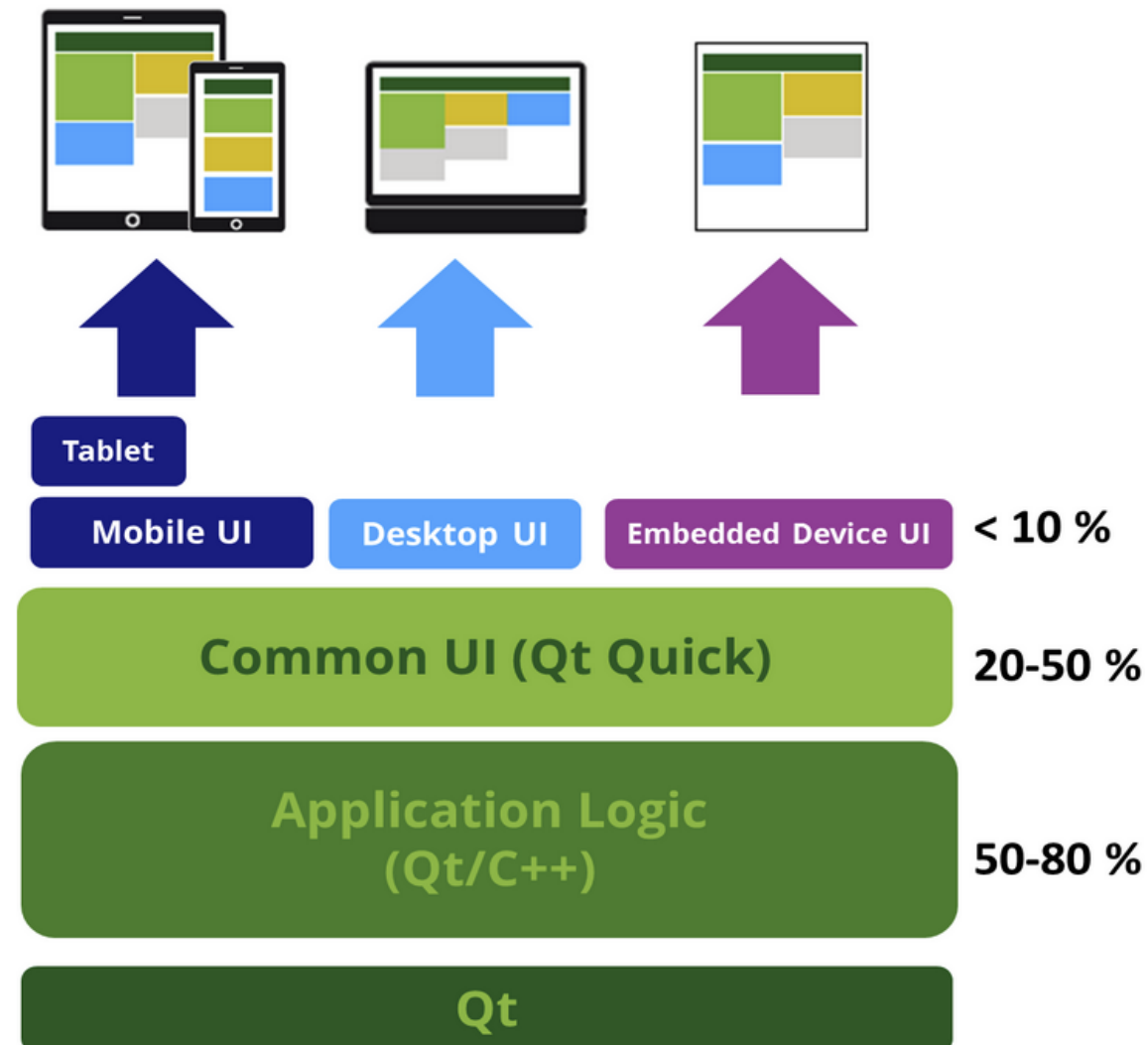
Window 类型概述

- 当用户试图关闭一个窗口时，会发射 closing 信号。你可以通过编写一个 onClosing 处理器并设置 `close.accepted = false`，强制窗口保持打开状态。
- 默认情况下，Window 是非模态显示的，但可以设置 `modality` 属性使其作为模态的。
- Window 的主要 property : `title`、`visibility`、`visible`、`flags` 等等。

<https://doc.qt.io/qt-6/qml-qtquick-window.htm>
|

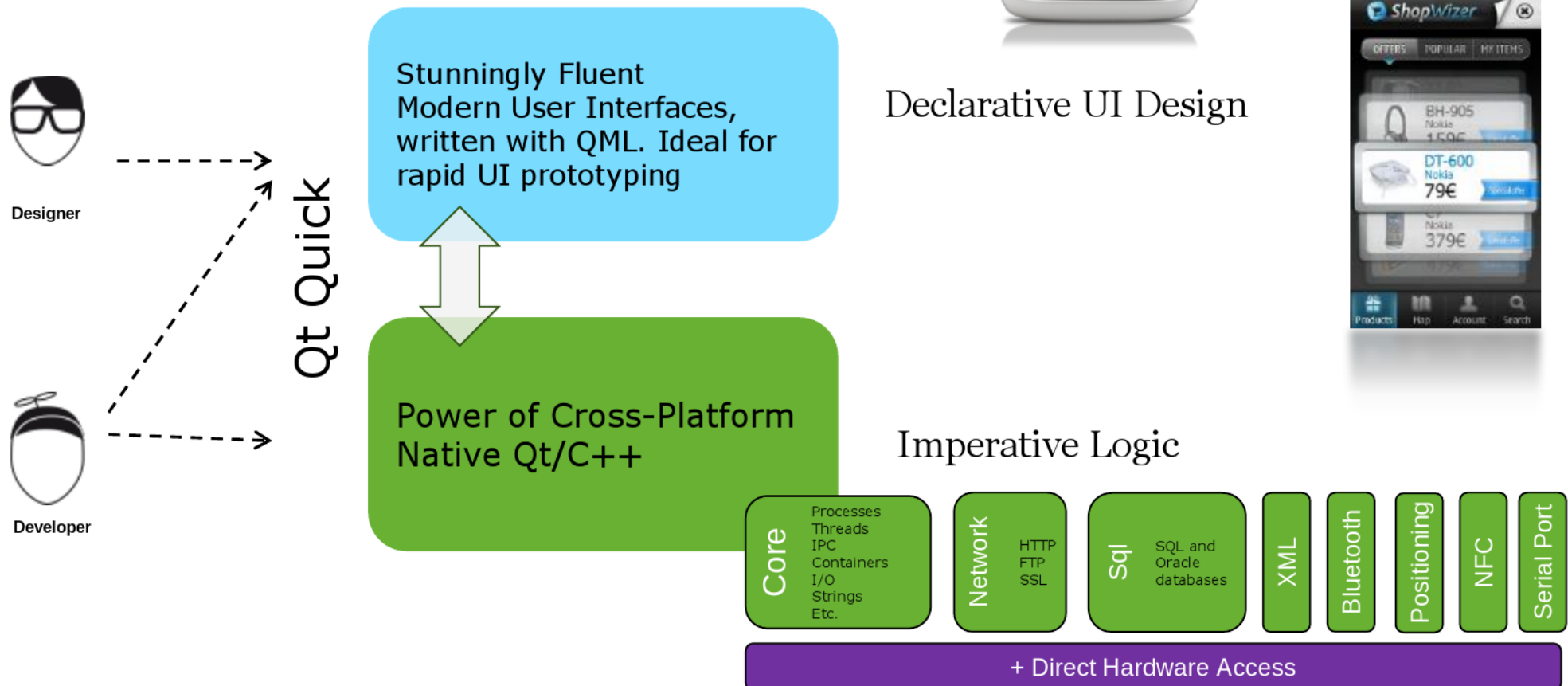
Qt6 应用架构

Scalable UI Design



UI 与 Logic 的分离

Rapid Workflow with Qt Quick



Create Your First Applications

To Develop With	Tutorials	Examples
Qt Widgets	> Text Finder > Notepad	Qt Widgets Examples
Qt Quick	> Qt Creator: Transitions > Qt Creator: Alarms > Qt Design Studio: Log In UI	Qt Quick Examples and Tutorials
Python	Qt for Python Tutorials	Qt for Python Examples

To Develop For	Tutorials	Examples
Mobile Devices	Qt Creator: Accelbubble	Mobile Examples
Embedded Devices	> Tutorial: Deploying Your First Project with Boot to Qt > Qt for Android Automotive Quick Starter Guide	> Examples in Qt for Android Automotive
MCUs	Qt Design Studio: Washing Machine UI	Qt Quick Ultralite Examples

<https://doc.qt.io/qt-6/create-your-first-applications.html>